

Claude & Codex

Two platforms, one mental model for the team — April 2026.

Why this briefing exists

We researched Claude and Codex so the team shares one mental model — not to decide a migration.

WHAT WE LOOKED AT

- Capability surface
- Quality & benchmarks
- Pricing economics
- Dual-tool feasibility
- Porter understanding

WHAT THIS IS *NOT*

- A vendor recommendation
- A commitment to switch
- A porter build plan
- A migration timeline

WHO THIS IS FOR

Anyone on the team who reaches for an AI coding tool — any seniority, any stack.

SOURCES

Built from **110 URL sources**, **10 YouTube-review NotebookLM Q&A**, and live Codex CLI v0.124.0 inspection.

PART 1

Understanding the landscape

What Codex is today, how it compares head-to-head, and the non-obvious patterns behind the numbers.

The Codex product surface

"Codex" is four products under one brand — don't conflate them.

CLI

Local pair-coder

ENGINEERS · TERMINAL-NATIVE

- Lives in the terminal; fast feedback loop
- Hooks (6 events, stable in v0.124) + subagents
- Starlark `.rules` policy engine
- Per-agent model routing

APP · DESKTOP

GUI with Computer Use

DAY-TO-DAY DRIVER · MACOS & WINDOWS

- Parallel git worktrees — agent per thread
- Inline diff UI · click-to-comment on diffs
- In-app browser with click-to-comment
- Opens PDFs / spreadsheets natively

CLOUD

Delegated tasks over GitHub / Linear / Slack

ASYNC · DELEGATE SURFACE

- Multi-hour autonomous runs
- Dispatch from PR comments, issues, chat
- No local machine required

APP SERVER

JSON-RPC 2.0 embedding protocol

EMBED IN YOUR OWN PRODUCT

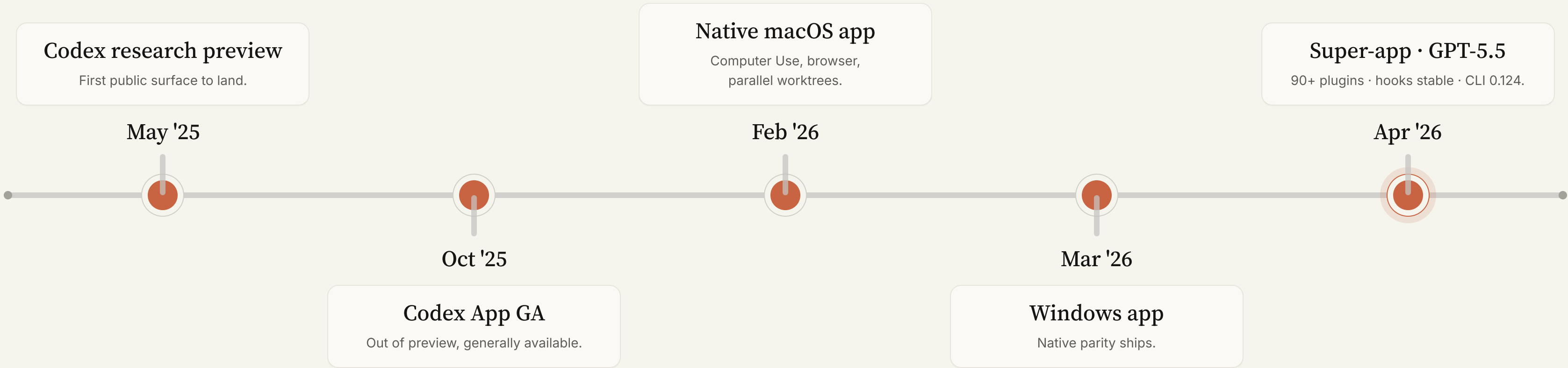
- Transports: stdio + WebSocket
- Thread / turn / item primitives
- Approval callbacks · filesystem API



CLI ≠ App ≠ Cloud — keep them *straight*.

Codex's 6-month ramp

The landscape moved fast. Our research is a snapshot.



↻ Re-verify *quarterly*. Six months is a different product.

Codex is not (just) a coding tool

2025 was a coding assistant; 2026 is the ambient-agent pitch — "a body for AI."

COMPUTER USE

Visual control of real Mac apps

MACOS GA · APR 2026

- Invoke via `@Computer Use` or `@AppName` ; per-app allow-list
- Independent cursor — runs while you keep working
- Cannot touch Terminal, Codex itself, or admin auth
- Trade-off vs Claude: higher fidelity, higher blast radius

CHRONICLE

Ambient memory from screen capture

PRO-ONLY · NO CLAUDE ANALOG

- Selected frames OCR'd server-side into markdown memory
- Frames auto-delete after 6h; memory at `$CODEX_HOME/.../chronicle/`
- Opt-in only; not available in EU / UK / CH
- Prompt-injection risk — pause for sensitive work

AUTOMATIONS + CLOUD

Schedule it, delegate it, walk away

ASYNC · NO LOCAL MACHINE

- Cron-like recurring agent jobs, kicked off from chat or app
- Multi-hour autonomous runs in OpenAI's cloud
- Dispatch from PR comments, Linear issues, Slack threads
- Monitor + resume from the ChatGPT iOS app

APPS SDK + IN-APP BROWSER

A platform, not just an editor companion

90+ FIRST-PARTY PLUGINS

- Jira, Linear, GitLab, Microsoft 365, Playwright, Figma...
- Click-to-comment in-app browser → agent loop on a dev server
- Codex Security: continuous per-commit vuln scanning (separate SKU)
- Remote SSH connections (alpha) — drive a cloud dev box locally



Evaluate Codex as a *desktop agent platform* — not as a Claude Code clone.

Capability matrix — primary tooling

CAPABILITY	CODEX	CLAUDE CODE	NOTES
CLI	✓	✓	Both polished; Claude's is more feature-rich
Desktop app	✓ best-in-class	~ buggy per reviewers	See Developer ergonomics
Cloud delegation	✓	✗	No first-party Claude analog
Computer Use	✓ visual, background	~ hijacks cursor, slower	—
In-app browser	✓ click-to-comment	✗	—
Mobile dispatch	✗	✓ Desktop from phone	—
1M context	✗ 400K ceiling	✓ Opus 4.7 native	—
Long-session stability	✓ 2–3 weeks	~ breaks at 50–60% context	—
Parallel git worktrees	✓	~ manual	—
Voice coding	~ ChatGPT voice	~ Desktop voice	—

Capability matrix — extensibility & ecosystem

CAPABILITY	CODEX	CLAUDE CODE
MCP client	✓	✓
MCP server	✓	✓
Skills	✓ SKILL.md layout	✓ SKILL.md layout
Plugins	Skills + MCP + apps	Skills + MCP + commands + hooks
Hooks	✓ 6 events · stable Apr 2026	✓ ~21 events · richer mutation
Subagents	✓ per-agent model routing	✓ per-agent model routing
ToolSearch (lazy tool loading)	✗	✓
Rules / policy engine	✓ Starlark <code>.rules</code>	~ settings.json permissions
Apps SDK	✓	✗
Projects / Artifacts	✗	✓ Claude.ai web

Quality & benchmarks

Roughly at parity — except on tool-calling.

SWE-bench Verified

2.6 pt gap · within noise



MCP-Atlas · tool-calling

• THE EXCEPTION



+9.2 pts

Claude's tool-calling moat — the strongest single axis where Codex hasn't caught up.

Long-context retrieval at 1M

qualitative · no defensible per-model number

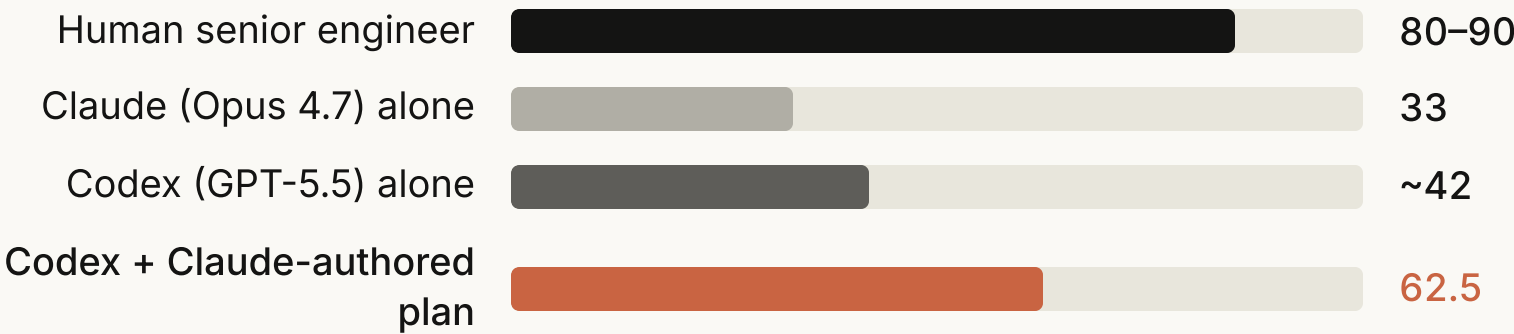
Treat qualitatively — context-window math, not benchmarks



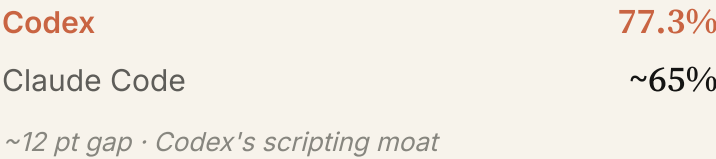
- ✓ Claude · 1M native
- ✓ Opus 4.7 regresses on MRCR v2
- ✓ Codex · 400K ceiling
- ✓ Tokenizer adds up to ~35%

Senior-engineer benchmark

First-principles refactor · higher is better

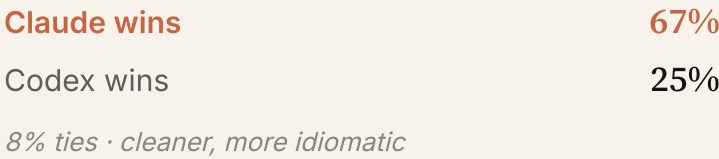


TERMINAL-BENCH 2.0 Agentic terminal tasks



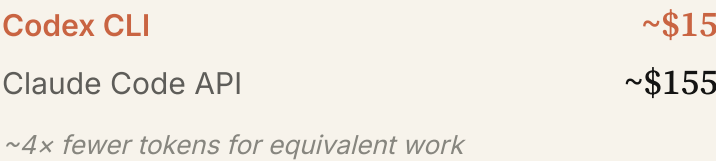
~12 pt gap · Codex's scripting moat

BLIND CODE-QUALITY PREFERENCE Dev review, head-to-head



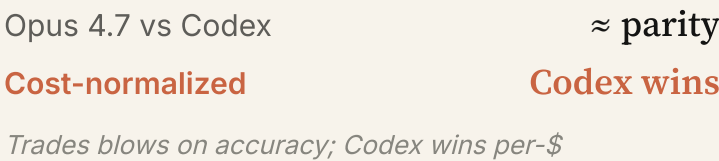
8% ties · cleaner, more idiomatic

TOKEN EFFICIENCY Same complex refactor



~4× fewer tokens for equivalent work

AIDER POLYGLOT Multi-language edit fidelity



Trades blows on accuracy; Codex wins per-\$

Pricing — subscription economics

The Standard / Premium gap is capacity, not feature access — 5× the price buys 5× the usage cap.

PLAN	ANNUAL / SEAT	MONTHLY / SEAT	MIN SEATS	CODING CLI INCLUDED
Claude Team Standard <i>1.25× Pro session usage</i>	\$20 per seat / month	\$25 per seat / month	5 min	✓ Claude Code
Claude Team Premium <i>6.25× Pro · separate Sonnet weekly cap</i>	\$100 per seat / month	\$125 per seat / month	5 min	✓ Claude Code · 5× volume
ChatGPT Business <i>renamed from "Team" · 2025-08-29</i>	\$20 per seat / month	\$25 per seat / month	2 min	✓ Codex
ChatGPT Enterprise <i>~\$60/seat typical · range \$45–\$75</i>	custom annual contract	—	150 min	⚠ Codex · unreachable at 10 seats

01 5× the price buys 5× the usage cap

Standard vs Premium · capacity, not features

- Both tiers ship Claude Code — Premium is **not** a feature unlock.
- Session usage: **1.25×** Pro (Standard) vs **6.25×** ($\approx 5\times$ Standard).
- Premium adds a **separate Sonnet weekly cap** — pure headroom.

02 Dual-tool is a rounding error

10 seats · annual list

- Business + Standard $\approx$ **\$4.8k/yr.**
- Business + Premium $\approx$ **\$14.4k/yr** — both CLIs, full volume.
- $\sim$ **0.5–1%** of an engineer's loaded cost.

03 "ChatGPT Team" is gone

Renamed to Business · 2025-08-29

- Team $\rightarrow$ **Business**; no separate Team SKU.
- Sub-150-seat orgs: Business is the **only** option.
- Enterprise floor: **150 seats** $\approx$ \$108k/yr.

04 \$20 / \$100 is list, not a deal

Anthropic published rates · annual

- Annual list: **\$20 / \$100** per seat.
- Monthly: **\$25 / \$125** — flat $\sim$ 20% saving.
- Treat as canonical Claude baseline.

Pricing — API token economics

At flagship, the long "Claude is more expensive per token" story inverts against current-gen OpenAI.

MODEL	INPUT / MTOK	CACHED / MTOK	OUTPUT / MTOK	VENDOR & ROLE
gpt-5.5 <i>launched 2026-04-23 · exactly 2× GPT-5.4</i>	\$5.00 per MTok in	<i>unverified</i>	\$30.00 2× per MTok out	OpenAI · flagship
gpt-5.5-pro <i>max-quality agentic</i>	\$30.00 per MTok in	<i>unverified</i>	\$180.00 per MTok out	OpenAI · pro tier
gpt-5.4 <i>Codex default · mid-generation baseline</i>	\$2.50 per MTok in	\$0.25 10% mult	\$15.00 per MTok out	OpenAI · workhorse
gpt-5.3-codex <i>Codex-routed only · ~17× over standard API</i>	\$43.75 per MTok in	\$4.38 10% mult	\$350.00 per MTok out	Codex SKU only
claude-opus-4-7 <i>flagship · 1M native</i>	\$5.00 per MTok in	\$0.50 10% mult	\$25.00 per MTok out	Anthropic · flagship
claude-sonnet-4-6 <i>balanced workhorse</i>	\$3.00 per MTok in	\$0.30 10% mult	\$15.00 per MTok out	Anthropic · mid
claude-haiku-4-5 <i>fast & cheap · 200K context</i>	\$1.00 per MTok in	\$0.10 10% mult	\$5.00 per MTok out	Anthropic · light

01 The GPT-5.5 inversion
First per-token raise in the 5.x cycle

- GPT-5.5 input = **Opus 4.7** at **\$5**.
- GPT-5.5 output **20% above** Opus 4.7 (\$30 vs \$25).
- "Claude costs more per token" **flips** at flagship.

02 Token-economy offset
Headline rate ≠ real cost

- OpenAI: 5.5 uses **materially fewer** output tokens.
- Per-task cost can land **below** GPT-5.4 despite 2× rate.
- Inversion holds on rate; **reverses** on agentic tasks.

03 Opus 4.7 tokenizer tax
Headline flat, effective up

- New tokenizer can emit **+35% tokens** per text.
- Effective Opus input: **up to ~\$6.75**; output **~\$33.75**.
- Worst-case content erases the headline parity.

04 Don't reach for Codex SKU
Standard API is ~17× cheaper

- gpt-5.3-codex** : **\$43.75 / \$350** per MTok.
- gpt-5.4** : **\$2.50 / \$15** for raw API use.
- Both vendors offer **50% batch discount**; cache reads **10×** off.

Two opposite failure modes

Know which one is biting before you blame the tool.

CLAUDE

Rate-limits & context rot

HALLUCINATES · FORGETS

- Context breakdown at **50–60%** usage → session clear
- Rate-limited in **45–60 min** on Max (\$100/mo) with 3–5 parallel sub-agents
- Peak-hour throttling: 5–11 AM PT · 1–7 PM GMT
- Opus 4.7 "rush-to-completion" — fabricates SHAs, package names
- Deny-rule bypass at >50 subcommands (CVE-class · fixed v2.1.90)

Symptom: *hallucinated file paths, forgotten decisions, defends hallucinations across 10 turns.*

CODEX

Visual-UI & long-run edge cases

STALLS · MISCLICKS

- Single session stable for **2–3 weeks** — biggest durability win
- OpenAI runs **10×–40×** usage promos on paid tiers
- Computer Use unreliable on unexposed UI (timer-button demo failed; music demo played wrong band)
- Auto-compression fails to trigger → context exhaustion on long runs
- Infinite-loop passivity — countdown idled on "10" for ~4 min, no self-correction

Symptom: *visual-DOM mismatches, keyboard-shortcut guessing, session stalls, terminal hangs.*

● SPOTLIGHT · CACHE & TTL ECONOMICS

When the cache silently breaks, the bill compounds

Anthropic exposes the cache TTL knob; OpenAI hides it — same failure mode, opposite visibility.

ANTHROPIC · PERNICIOUS AMNESIA

2026-03-06 → today

Sub-agent prompt-cache TTL silently dropped **1 h** → **5 min**. Every idle gap re-writes the prefix at **12.5×** read cost.

- **+17–32%** cost overhead on long sessions; one repro logged \$6.28 → \$15.54 daily
- Sub-agents only — main agent stayed at 1 h; direct-API impact unverified
- Anthropic postmortemed the adjacent thinking-history bug (Apr 23) — never *this* change
- Audit: `grep ephemeral_5m_input_tokens ~/.claude/projects/**/*.*.jsonl`

OPENAI · SYMMETRIC RISK

No public regression to date

Auto-caches any prefix ≥ 1024 tokens — **no TTL knob** exposed via Codex CLI. Same failure would be quieter, not absent.

- Failure mode = cache-hit rate quietly drifts; no contract to compare against
- Anthropic's incident took 8 days to detect; an OpenAI equivalent could take weeks
- Token-burn issue (Github Issue #14593): 20% of quota burnt in 2h on simple prompts
- Audit: track `usage.prompt_tokens_details.cached_tokens` ; alert on >20% W/W drop

Day-to-day ergonomics

What's actually nice to use — once you're past the benchmarks.

CODEx

Strongest in the desktop

GUI · DIFFS · BROWSER

- **Parallel git worktrees** — agent per thread, no conflicts
- **Inline diff UI** with click-to-comment on individual hunks
- **In-app browser** — click UI, type a comment, agent gets visual context
- Native open for PDFs, spreadsheets, images alongside the diff

Tone: dry, terminal-first — "belt and suspenders." Reach for it on a desktop-heavy day.

CLAUDE

Strongest in the CLI & on mobile

SLASH · REMOTE · DISPATCH

- **Co-work mode** — desktop surface for multi-step knowledge work
- **Mobile dispatch** — assign tasks to your desktop agent from your phone
- `/remote-control` — drive a CLI session from any device
- Dense slash catalog: `/autofix`, `/branch`, `/doctor`, `/loop`

Tone: warm, dynamic spinner verbs, friendlier defaults. Reach for it on a terminal- or phone-heavy day.

• SPOTLIGHT · REASONING-EFFORT CEILING

When you crank the dial all the way up, only one of them keeps going

Codex caps the effort knob at `xhigh`; Opus 4.7 exposes a `max` tier above it — quiet difference, loud consequences on the hardest problems.

CODEx · CEILING = XHIGH

CLI flag · `--reasoning xhigh`

Top of the dial is **xhigh**. Strong on most tasks, but **no escape hatch** when a problem genuinely needs more thinking budget.

- Most day-to-day work lands at `medium` / `high` — ceiling rarely matters
- On gnarly refactors, you hit the wall and start splitting the task by hand
- No public roadmap for a higher tier; ceiling has held since GA

OPUS 4.7 · CEILING = MAX

CLI flag · `--effort max`

One step above `xhigh`: **max** burns longer and budgets more thinking tokens for problems that won't yield otherwise.

- Reserved for the 5–10% of tasks that visibly stall on lower tiers
- Cost & latency scale — not a default, but the difference between solved and abandoned
- Practical use: hard root-cause hunts, multi-file refactors, ambiguous specs

Per-token vs per-task inversion

Intuition is wrong in both directions on API spend — the headline rate and the real bill point opposite ways.

1 RAW PER-TOKEN

Claude looks cheaper on the rate card

- `claude-opus-4-7` · \$5 in / \$25 out per MTok
- `gpt-5.5` · \$5 in / \$30 out per MTok — Apr 23 launch
- GPT-5.5 is **exactly 2× GPT-5.4** (\$2.50 / \$15) — first explicit per-token raise of the 5.x cycle
- Output gap: Opus is **~17% cheaper** per output token at face value
- Cache-read math is symmetric — both vendors at 10× discount on hits

2 TOKENIZER CORRECTION

Opus 4.7's new tokenizer inflates the bill

- Up to **+35% more tokens** for the same source text (released 2026-04-16)
- Effective rate climbs to **up to \$6.75 in / \$33.75 out**
- Worst-case content: Claude becomes **~12% more expensive** per useful output unit than GPT-5.5
- Inflation varies by content type — code & non-English text hit hardest
- Headline rate didn't change vs Opus 4.6 — the cost increase is invisible until you reconcile bills

3 PER-TASK REALITY

Codex finishes the job in far fewer tokens

- Ecosystem-sim test: GPT-5.5 used **28K** output tokens vs Opus 4.7's **250K** on the same task — **~9× fewer**
- Across 4 coding experiments: GPT-5.5 **~\$3 cheaper** overall; Claude won by ~\$1 on one (solar-system build)
- Real refactor anecdote: Codex CLI **~\$15** vs Claude Code API **~\$155** — **~10× per-task gap**
- Published SWE-bench cost: Opus 4.7 **\$0.71** / fix vs GPT-5.3-Codex **\$0.06** — 12× spread
- Direction holds; magnitude is task-dependent — not a benchmark

 Compare on *cost-per-completed-task*, not *cost-per-token*.

Different bets across the portfolio

Same developer audience, opposite convictions about which capabilities matter most.

CAPABILITY	CODEX / OPENAI	CLAUDE / ANTHROPIC
Coding model <i>training emphasis</i>	Long-horizon agent shape · trained on agent traces	General coding LM tuned for short-horizon dev work
Image generation <i>raster output</i>	gpt-image-1 / 1.5 · DALL·E 3 sunset 2026-05-12	× <i>none first-party</i> — third-party via MCP only
Video generation <i>text-to-video</i>	Sora 2 / 2 Pro · API sunset 2026-09-24	× <i>none</i> — no roadmap announced
Vision / understanding <i>image & PDF input</i>	GPT-4V — multimodal input across the stack	Vision <i>read-only</i> — analyzes input but never emits raster
Realtime voice <i>speech ↔ speech</i>	gpt-realtime / -mini · SIP / phone, MCP-aware	Push-to-talk only · partner ASR/TTS pipeline
Speech ↔ text <i>STT + TTS</i>	gpt-4o-transcribe · gpt-4o-mini-tts	× <i>none first-party</i> — assemble third-party
Embeddings & moderation <i>retrieval + safety</i>	text-embedding-3 · omni-moderation	× <i>none</i> — bring-your-own embeddings
Open weights <i>self-host option</i>	gpt-oss — open-weights, self-host eligible	× <i>closed-source only</i>
Artifacts <i>static + live</i>	ChatGPT Canvas — single-doc live editing; no connector-backed dashboards	Live Artifacts in Cowork — auto-refresh, connector-backed dashboards
Design surface <i>first-party</i>	Figma partnership · MCP round-trip; no native app	Claude Design — Opus-powered, codebase-native

01

Eight specialty models vs zero

Bundled multimodality vs single-purpose stack

- OpenAI ships **image, video, STT, TTS, realtime voice, embeddings, moderation, open-weights** — eight first-party specialty models.
- Anthropic ships **vision input only** — every other modality routes through MCP / partners.
- Two are **actively sunsetting**: DALL·E 3 (May 12) and Sora 2 API (Sept 24).

02

The focus is paying off

Revenue mix · March 2026

- Anthropic **\$30B ARR** · ~80% enterprise · **passed OpenAI** on revenue.
- OpenAI **\$25B ARR** · ~40% enterprise (Sacra projects parity by EOY).
- Claude Code at **\$2.5B run-rate** · 54% enterprise coding share vs OpenAI's 21%.

03

"Why no Anthropic image-gen?"

Inferred motive · not on-record

- No on-record statement; analyst inference from **hiring, revenue mix, product cadence**.
- Image / video carry extra **safety surface** (deepfakes, CSAM, copyright) Anthropic hasn't committed to.
- Manifold prediction markets **still open** on a 2026 first-party image model.

Reach for Codex / Reach for Claude

When both are available, the shape of the work — not habit — should decide which agent you open first.

REACH FOR CODEX

Long-horizon, multi-modal work

Reach for Codex when the task runs longer than a single sitting, fans out across repositories, or needs a first-party specialty model the Anthropic stack does not ship.

- long-running async work, parallel worktrees
- image / video / TTS / STT / embeddings — eight first-party specialty models
- open-weights via gpt-oss for self-hosted reasoning
- cleaner sandboxed "from-scratch CLI" tasks

REACH FOR CLAUDE

Short loops, mature ecosystem

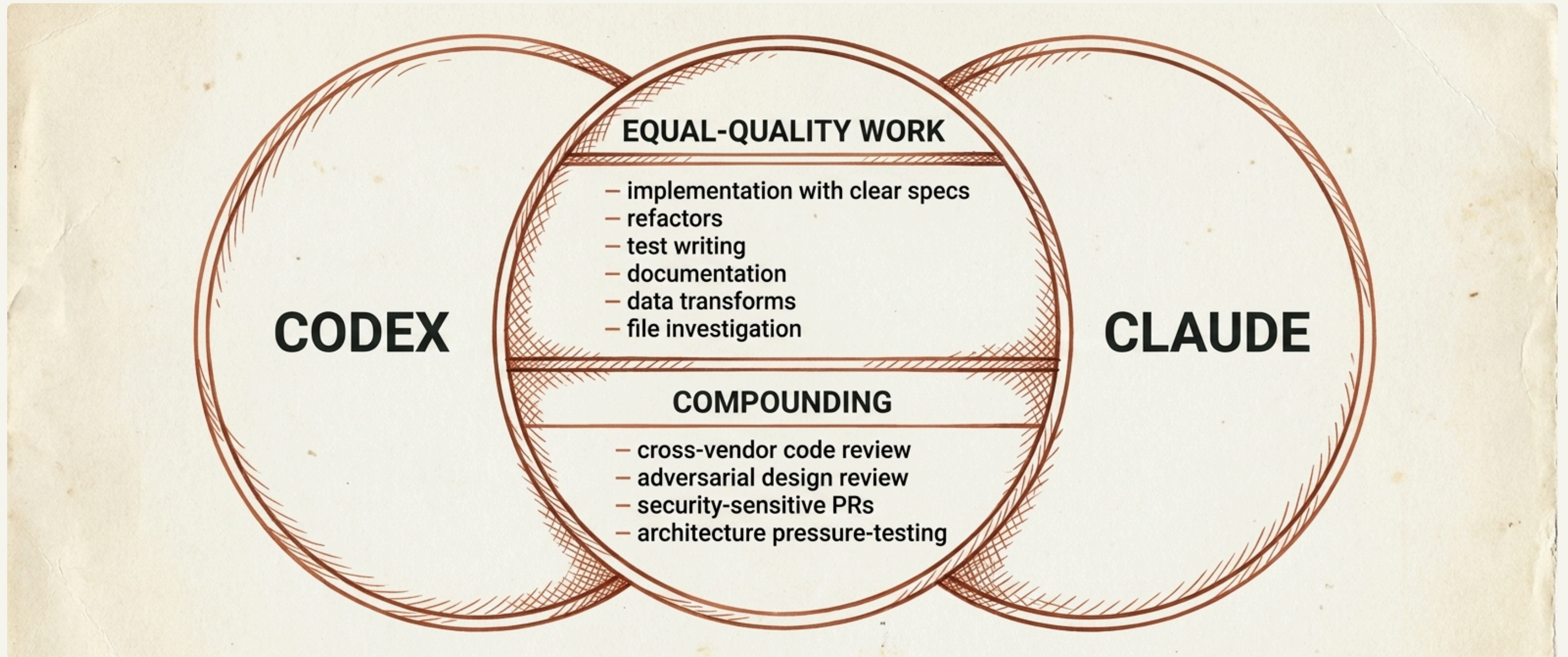
Reach for Claude when the task is conversational, depends on dense connector / hook tooling, or benefits from the more mature plugin and skill surface.

- 1M context · cost-efficient short loops · prompt-cache economics
- more mature plugins, skills, hooks — five primitives vs three
- ToolSearch + denser MCP catalog · single-loop computer use & browser automation
- mobile dispatch · Cowork · Live Artifacts
- tool-calling lead (MCP-Atlas: Opus 4.7 77.3% vs GPT-5.4 68.1%)

 Default to *Claude* — reach for *Codex* when the work shape rewards async cloud delegation.

The overlap zone

Don't agonize — and know when running both pays off.



PART 2

Working with both

Part 1 mapped the landscape. Part 2 turns it into a working playbook.

Three dual-tool patterns that work

Not theory — these are the three shapes teams actually ship with.

DT-1

Author with one, review with the other

Author
primary agent

→

Cross-review
other vendor

→

Merge
to main

- The author model misses the bugs *it* would write
- Cross-vendor review catches what same-vendor review normalizes
- Use on non-trivial PRs, security-sensitive changes, generated code
- Skip on doc-only edits and time-critical reverts

DT-2

Specialist routing

LONG

Long-horizon

→

Codex

SHORT

Short loops

→

Claude

- Match the agent to the work, not the habit
- Codex: multi-file refactors, long autonomous runs, backend-heavy
- Claude: dense conversational iteration, UI work, tool-calling
- Heuristic — if you'd start a timer and walk away, it's Codex

DT-3

Background-vs-foreground split

Codex async
cloud job

→

Claude IDE
foreground

- Long cloud job runs while immediate work continues in the IDE
- Neither blocks the other
- Earns its keep when async > 10 min and foreground is independent
- Concurrent edits on shared files need worktree isolation

⚡ Pick by *work shape*, not habit — that's the dual-tool dividend.

Cost — a \$40–120 / dev / mo conversation

Five postures, one table — from solo minimums to the full dual-tool stack.

POSTURE	SETUP	10 SEATS × 12 MO	PER DEV / MO	INCLUDES
Pure Claude solo	Claude Pro × 1 (annual)	\$2,040	\$17	Claude Code on Pro; 200k context; ~44k tokens / 5h cap
Pure Codex solo	ChatGPT Plus × 1	\$2,400	\$20	Codex CLI on Plus; 400k context ; 5h rolling + weekly cap
Dual-tool minimum	Claude Pro + ChatGPT Plus	\$4,440	\$37	Both CLIs at lowest paid tier ; no team admin controls
Dual-tool typical	Team Standard + Business (annual)	\$4,800	\$40	Claude Code + Codex; team admin ; 5-seat and 2-seat minimums met
Premium dual-tool	Team Premium + Business (annual)	\$14,400	\$120	6.25× Pro session volume (Claude) + Business pool (Codex); upper bound



Cost reality at a glance

 Team scale lands at **\$40–120 / dev / mo** by tier mix

 Solo postures are cheaper but skip cross-vendor compounding

 Enterprise SKUs excluded — published rates only

 Even Premium dual-tool is **<1%** of an engineer's loaded cost.

Operational risk register

Running two tools means running two operational surfaces — and two places for things to go wrong.

RISK	LIKELIHOOD	IMPACT	MITIGATION
Vendor lock-in via tool-specific assets	Medium	Medium	Quarterly review of skill-library export coverage
Model-version drift erodes cross-review value	Medium	High	Pin known-good model snapshots per vendor
Cache-pricing contract changes silently	Medium	High	Monitor cache-hit ratio in usage logs weekly
Operational complexity from two CLIs	High	Medium	Single onboarding doc covering both CLIs
Knowledge fragmentation across team	Medium	Medium	Rotate cross-vendor pairing assignments monthly
Cost creep from idle subscriptions	Medium	Medium	Tie seat renewals to per-tool usage review
Audit gaps when AI-driven changes lack provenance	Low	High	Vendor tag in every commit trailer by default

01 Lock-in is an exit cost that grows silently

Skills · plugins · commands · hooks

- Each tool-specific asset accumulates value that **only ports with effort**.
- The bill for switching shows up **only when you switch** — until then it stays invisible.
- Quarterly export-coverage review keeps the ledger honest.

02 Contract drift hits cost & quality together

Model rev · cache pricing

- Vendors change the contract **without notice** — and shared failure mode.
- First signal is a **spike in cost** or a **drop in review quality**, not a release note.
- Pin model snapshots; track cache-hit ratio weekly.

03 Two CLIs compound every other risk

Auth · config · cost surfaces

- Two **auth trails**, two **config surfaces**, two **cost meters**.
- Each surface multiplies the chance of a quiet drift going unnoticed.
- Single onboarding doc + tagged commit trailers blunt the blast radius.

When dual-tool makes sense

Three qualifying questions: team size, workload shape, and convention discipline.

1 TEAM SIZE

Five-plus engineers, regular cadence

- 5 or more developers writing code daily
- Enough concurrent work to keep both tools occupied at their standard tier
- Cross-vendor PR review reaches multiple PRs per week
- Onboarding overhead amortizes across multiple new hires per year

2 WORKLOAD BREADTH

Mixed shape — short loops *and* long horizons

- Mix of short conversational fixes and long-horizon autonomous runs
- Work regularly touches both backend systems and UI surfaces
- Security-sensitive PRs that warrant a cross-vendor second opinion
- Async cloud delegation is already part of the team's normal rhythm

3 CONVENTION TOLERANCE

Discipline to hold two setups without friction

- The team can hold two slightly different tool setups without friction
- Engineers pick the right tool by task shape, not by habit
- A documented pattern exists for "this PR needs a second opinion"
- Documentation discipline is strong enough to record per-tool decisions

⊘ When *not* to bother — solo dev, single workload shape, tight budget.

Signals to ramp up / ramp down

Watch these metrics, not the hype cycle — the data tells you when to shift.

RAMP UP WHEN

Signals it's working

- Cross-vendor review catches a recurring bug class that same-vendor review normalized
- Two parallel agent runs ship a feature in half the calendar time of one sequential run
- A senior engineer notes a missed edge case that only the second pass surfaced
- DT-2 specialist routing settles into team muscle memory without a decision framework
- Onboarding doc gets cited in PR reviews more often than the source-of-truth wiki
- A new hire reaches productive output in week 2 instead of week 4

RAMP DOWN WHEN

Signals it's not

- Cost-per-merged-PR doubles over a quarter without a measurable quality lift
- Junior engineers still can't articulate which tool to reach for after 90 days on the team
- Cross-review notes cluster on the same surface repeatedly — diminishing returns
- A model version update changes review behavior silently; recalibration loops eat the savings
- Audit log queries span two tools and take more than a day to reconstruct a change
- Quarterly check finds the second tool sat idle for the majority of the quarter

Re-evaluate quarterly, not on hype cycles.

Porter — what ports / what needs adaptation

Moving between the two tools costs effort — here is where that effort concentrates.

ITEM	CLAUDE FORM	CODEX FORM	EFFORT	STATUS
System prompts	<code>system_prompt</code> at thread start	<code>developer_instructions</code> at agent start	Trivial	Clean
Plugin manifests	<code>.claude-plugin/plugin.json</code> (thin, 4–5 keys)	<code>.codex-plugin/plugin.json</code> (same shape, add <code>skills</code> pointer)	Trivial	Clean
MCP servers	<code>.mcp.json</code> at project root or <code>settings.json</code>	<code>[mcp_servers.*]</code> in <code>config.toml</code> or plugin-root <code>.mcp.json</code>	Trivial	Clean
Skills	<code>skills/<n>/SKILL.md</code> — up to 6 frontmatter keys	<code>skills/<n>/SKILL.md</code> — 2 frontmatter keys only	Adapt	Adapt
Sub-agents	<code>.claude/agents/*.md</code> — Markdown with YAML frontmatter	<code>.codex/agents/*.toml</code> — TOML with <code>developer_instructions</code> body	Adapt	Adapt
Settings / permissions	<code>.claude/settings.json</code> glob allow/deny rules	<code>.codex/config.toml</code> + sandbox mode; no per-skill enforcement	Adapt	Adapt
Hooks (portable events)	<code>SessionStart</code> , <code>UserPromptSubmit</code> , <code>Stop</code> — stable events	Same three events supported; JSON config at <code>.codex/hooks.json</code>	Adapt	Adapt
Task() dispatch pattern	<code>Task(subagent_type)</code> inline calls in skill bodies	Subagents invoked differently — every call site needs rewriting	Adapt	Adapt
Output styles	<code>.claude/output-styles/*.md</code> — named style files	No equivalent surface	Hard	None
Hooks (file-read matchers)	<code>PreToolUse</code> targets <code>Read Grep Edit</code> matchers	<code>PreToolUse</code> experimental, Bash-tool interception only	Block	None

Top friction surprises



- 🕒 `PreToolUse` on `Read|Grep|Edit` is the single hook that cannot port — file-read gating must move into scripts.
- 🕒 Every `Task(subagent_type)` call in skill bodies is a hand-rewrite — no mechanical transform covers it.
- 🕒 Skill frontmatter surgery (dropping 4+ Claude-only keys) is per-file toil that adds up across large plugin trees.

Open questions

Five threads to pull next quarter, not today.

Q1 · ENGINEERING OPS · FINANCE

When does cheaper just win?

If per-seat costs climb 20–50% over the next 12 months, does dual-tool still pencil out, or does the cheaper of the two win by default?

Q2 · ENGINEERING QUALITY

Will the patterns hold?

As both tools roll model versions independently, do the cross-vendor review patterns still work, or does behavioral drift erode their value?

Q3 · ARCHITECTURE · VENDOR RISK

What does swap cost in 18 months?

Skills, plugins, and prompt assets are tool-specific today. What's the migration cost if we need to consolidate to one vendor later?

Q4 · ORG · HIRING

Compound or fragment?

Does running both tools genuinely compound team learning, or does it fragment expertise across two ecosystems newer engineers can't master fluently?

Q5 · ENGINEERING OPS

When do we ramp down?

At what team size or workload mix does dual-tool become net-negative — when overhead exceeds compounding gains?

These need owners, not answers — yet.



- ✓ Re-test in six months — not today
- ✓ Each thread needs a named owner
- ✓ Track signals, not opinions

Source documents & links

Where to read further — primary sources and independent coverage.

VENDOR

Official docs

- OpenAI Codex docs — developers.openai.com/codex
- Claude Code docs — code.claude.com
- OpenAI Codex landing — openai.com/codex
- Claude pricing — claude.com/pricing
- Claude support: Team plan — support.claude.com/.../team-plan
- Anthropic news (Opus 4.7) — anthropic.com/news/claude-opus-4-7

OPEN SOURCE

GitHub

- OpenAI Codex repo — github.com/openai/codex
- OpenAI Codex plugin for Claude Code — github.com/openai/codex-plugin-cc
- Anthropic plugin marketplace — github.com/anthropics/claude-plugins-official
- Anthropic Skills repo — github.com/anthropics/skills
- AGENTS.md spec — github.com/agentsmd/agents.md

COMMUNITY

Independent reference

- Simon Willison (Codex skills coverage) — simonwillison.net
- VS Code blog (multi-agent development) — code.visualstudio.com/blogs/.../multi-agent-development
- Daniel Vaughan (Codex CLI config reference) — codex.danielvaughan.com
- LevelUp (Claude Code mental model) — levelup.gitconnected.com/.../claude-code-mental-model
- AtCyrus (MCP ToolSearch context pollution) — atcyrus.com/stories/mcp-tool-search-...-guide
- Composio (How to MCP with Codex) — composio.dev/content/how-to-mcp-with-codex